# Module 10 - Exception Handling

---

## Table of Contents


- [ ] 1. What Is an Exception? — The Alarm Analogy
- [ ] 2. `try` / `except` — Catching the Error
- [ ] 3. Catching Specific Exception Types
- [ ] 4. Capturing the Error Message with `as e`
- [ ] 5. `else` — When Everything Goes Right
- [ ] 6. `finally` — Always Runs, No Matter What
- [ ] 7. `raise` — Triggering Exceptions Yourself
- [ ] 8. Custom Exceptions — Building Your Own Error Types
- [ ] 9. The Exception Hierarchy
- [ ] 10. Summary and Next Steps


---

## 1. What Is an Exception? — The Alarm Analogy

Imagine a SOC has a network scanner running 24/7. It tries to connect to hundreds of IPs. Sometimes a host is unreachable. Sometimes a port is closed. Sometimes the input is malformed.

Without exception handling, **the entire scanner crashes on the first problem** and stops scanning all remaining targets. One bad IP kills the whole job.

Exception handling is the equivalent of a **smoke detector with an automatic suppression system**: the alarm fires, the system responds, and the building keeps running.

```
WITHOUT handling:          WITH handling:
scan 10.0.0.1  OK          scan 10.0.0.1  OK
scan 10.0.0.2  CRASH!      scan 10.0.0.2  ERROR caught -> log it ->
continue
scan 10.0.0.3  (never runs)  scan 10.0.0.3  OK
```

### Key vocabulary

| Term | Plain English |
|---|---|
| **Exception** | An error that occurs while the program is running (runtime error) |
| **Raise** | Python 'throws' an exception when something goes wrong |

| Term | Plain English |
|---|---|
| **Handle** | Your code catches the exception and decides what to do |
| **Traceback** | The error report Python prints — shows what went wrong and where |

## What causes exceptions?

| Exception type | What triggers it |
|---|---|
| `ValueError` | Wrong type of value — `int('abc')` |
| `TypeError` | Wrong type altogether — `'hello' + 5` |
| `ZeroDivisionError` | Dividing by zero |
| `KeyError` | Accessing a dict key that doesn't exist |
| `IndexError` | Accessing a list index out of range |
| `FileNotFoundError` | Opening a file that doesn't exist |
| `ConnectionError` | Network connection failed |
| `PermissionError` | No access rights to a file or resource |

All of these will **crash your program** if you don't handle them.

In [ ]:
```
# What an unhandled exception looks like:
# Run this cell to see the crash — then we'll fix it.

raw_cvss = "not-a-number"   # imagine this came from a broken API response
cvss_score = float(raw_cvss)  # ValueError: could not convert string to float
print(f"CVSS score: {cvss_score}")
print("This line never runs.")
```

---

# 2. `try` / `except` — Catching the Error

The basic structure:

```
try:
    # code that might fail
except:
    # code that runs IF it fails
```

- Python **tries** to run the code in the `try` block
- If an exception occurs, Python **jumps immediately** to the `except` block
- Code after the failed line in the `try` block is **skipped**
- After the `except` block, **execution continues normally**

```
try block:                    except block:
  line 1 runs OK                skipped if try succeeded
  line 2 FAILS  ----------->    this runs instead
  line 3 skipped
                              code after try/except: always runs
```

## Warning — Bare `except` is an anti-pattern

```python
# BAD: catches everything including bugs you want to know about
except:
    print("something went wrong")

# GOOD: always name the exception type you expect
except ValueError:
    print("invalid value")
```

Catching everything silently hides real bugs. In security code this is especially dangerous — a swallowed exception might mean a connection silently failed and you never logged it.

In [1]:
```python
# Topic: Basic try/except — scanner survives a bad target

import socket

def check_port(target_ip, port):
    """Attempts to connect to a port. Returns True if open, False if closed/error."
    try:
        # socket.setdefaulttimeout limits how long we wait before giving up
        socket.setdefaulttimeout(1)
        sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        result = sock.connect_ex((target_ip, port))
        sock.close()
        return result == 0  # 0 means connected successfully
    except socket.gaierror:
        # Hostname/IP could not be resolved — bad target, skip it
        print(f"  [SKIP] Cannot resolve {target_ip}")
        return False


# A mix of valid and invalid targets
targets = [
    ("127.0.0.1", 80),
    ("not-a-real-host-xyz", 443),   # invalid hostname — would crash without try/ex
    ("127.0.0.1", 22),
]

for ip, port in targets:
    result = check_port(ip, port)
    print(f"{ip}:{port} -> {'OPEN' if result else 'CLOSED/FILTERED'}")

print("Scan complete.")   # this runs even after the error
```

```
127.0.0.1:80 -> CLOSED/FILTERED
  [SKIP] Cannot resolve not-a-real-host-xyz
not-a-real-host-xyz:443 -> CLOSED/FILTERED
127.0.0.1:22 -> OPEN
Scan complete.
```

## Predict

**Exercise 1 — Predict**

Before running the cell below, predict which lines will print and in what order. Write your prediction here:

```
Line A: 'Trying to connect...'
Line B: 'Connection failed'
Line C: 'Port scan finished'
```

Prediction (which lines print, in which order):

1. ?
2. ?
3. ?

In [ ]:
```python
# Predict the output before running!

open_ports = []

try:
    print("Trying to connect...")    # line A
    open_ports.append(int("eighty"))  # this will fail
    print("Appended successfully")    # does this run?
except ValueError:
    print("Connection failed")        # line B

print("Port scan finished")           # line C
print(f"Open ports collected: {open_ports}")
```

▶ 💡 **SOLUTION**

---

# 3. Catching Specific Exception Types

You can have **multiple `except` blocks**, one per exception type. Python checks them **top to bottom** and runs the first one that matches.

```python
try:
    risky_code()
except SpecificError:      # checked first
    handle_specific()
```

```
except OtherError:          # checked second
    handle_other()
except Exception:           # safety net — catches most things
    handle_unexpected()
```

You can also catch **multiple exception types in one block** using a tuple:

```
except (ValueError, TypeError):
    print("bad value or wrong type")
```

## Rule: most specific first

Always put the most specific exceptions at the top. If you put `Exception` first, it catches everything and the specific handlers below never run.

In [ ]:
```
# Topic: Multiple except blocks — realistic scanner input validation

def parse_cvss_score(raw_input):
    """Parse a CVSS score from a raw string (e.g. from a CSV or API response)."""
    try:
        score = float(raw_input)        # ValueError if not a number

        if not 0.0 <= score <= 10.0:    # CVSS range is always 0.0-10.0
            raise ValueError(f"CVSS score {score} is out of range (0.0-10.0)")

        return score

    except ValueError as e:
        print(f"[PARSE ERROR] Invalid CVSS input '{raw_input}': {e}")
        return None
    except TypeError:
        # raw_input was None — e.g. missing field in the source data
        print(f"[PARSE ERROR] CVSS field is missing (None received)")
        return None


# Simulate parsing several rows from a vulnerability export
raw_scores = ["9.8", "not_a_score", None, "15.0", "7.5"]

parsed = []
for raw in raw_scores:
    result = parse_cvss_score(raw)
    if result is not None:
        parsed.append(result)

print(f"\nSuccessfully parsed: {parsed}")
```

## Practice

### Exercise 2 — Write

Write a function `parse_log_line(line)` that:

1. Splits the log line on `'|'` and expects exactly 3 parts: `source_ip`, `event_type`, `severity`
2. Returns a dict `{'source_ip': ..., 'event_type': ..., 'severity': ...}`
3. Raises a `ValueError` if the line doesn't have exactly 3 parts
4. Wraps the logic in `try/except` — if a `ValueError` occurs, print an error message and return `None`

Test with these lines:

- `'10.0.0.14|FAILED_LOGIN|HIGH'` — valid
- `'192.168.1.5|PORT_SCAN'` — only 2 parts — invalid
- `''` — empty — invalid

In [ ]:
```
# Your code here
```

▶ 💡 **SOLUTION**

**Exercise 3 — Fix**

The function below is supposed to look up a service name from a port number. It has two bugs in the exception handling. Find and fix them both.

```
def get_service_name(port_services, port_number):
    try:
        port_number = int(port_number)
        return port_services[port_number]
    except KeyError, ValueError:       # Bug 1
        print(f"Port {port_number} not found")
        return None
    except:
        print(f"Unknown error")
    finally
        print("Lookup done")           # Bug 2
```

In [ ]:
```
# Fix the buggy code:

def get_service_name(port_services, port_number):
    try:
        port_number = int(port_number)
        return port_services[port_number]
    except KeyError, ValueError:       # Bug 1
        print(f"Port {port_number} not found")
        return None
    except:
        print(f"Unknown error")
    finally
        print("Lookup done")           # Bug 2


services = {22: "SSH", 80: "HTTP", 443: "HTTPS", 3306: "MySQL"}
```

```
print(get_service_name(services, 443))
print(get_service_name(services, 9999))
```

▶ 💡 **SOLUTION**

---

## 4. Capturing the Error Message with `as e`

Adding `as e` after the exception type gives you access to the actual exception object, including its message. This is essential for logging.

```
except ValueError as e:
    print(f"Error: {e}")        # prints the exception message
    print(type(e).__name__)      # prints 'ValueError'
```

In security tools you almost always want `as e` — you need to log *what* went wrong, not just *that* something went wrong.

```
BAD log:   [ERROR] Scan failed
GOOD log:  [ERROR] Scan failed — ConnectionRefusedError: [Errno
111] Connection refused (10.0.0.5:443)
```

The GOOD log tells an analyst exactly what happened and where.

In [ ]:
```
# Topic: 'as e' — capturing exception details for structured logging

import socket

def scan_port_logged(target_ip, port):
    """Scan a single port and return a structured result dict."""
    result = {
        "target": f"{target_ip}:{port}",
        "status": None,
        "error": None
    }

    try:
        socket.setdefaulttimeout(0.5)
        sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        code = sock.connect_ex((target_ip, port))
        sock.close()
        result["status"] = "OPEN" if code == 0 else "CLOSED"

    except socket.gaierror as e:
        # Name resolution failed — log the specific reason
        result["status"] = "ERROR"
        result["error"] = f"DNS resolution failed: {e}"

    except socket.timeout as e:
        # No response within the timeout window — likely filtered by a firewall
        result["status"] = "FILTERED"
```

```
        result["error"] = f"Timeout: {e}"

    except OSError as e:
        # General OS-level network error
        result["status"] = "ERROR"
        result["error"] = f"OS error: {e}"

    return result


targets = [("127.0.0.1", 80), ("bad-host-xyz", 443)]

for ip, port in targets:
    r = scan_port_logged(ip, port)
    if r["error"]:
        print(f"[{r['status']}] {r['target']} — {r['error']}")
    else:
        print(f"[{r['status']}] {r['target']}")
```

## Practice

**Exercise 4 — Write**

Write a function `load_blocklist(filepath)` that:

1. Opens a file and reads its lines (you can simulate with `open(filepath).readlines()`)
2. Returns the list of lines stripped of whitespace
3. Catches `FileNotFoundError as e` — logs the error and returns an empty list
4. Catches `PermissionError as e` — logs it and returns an empty list

The error log must include the exception message using `as e`.

Test by calling it with a filename that doesn't exist: `load_blocklist('blocked_ips.txt')`.

In [ ]:
```
# Your code here
```

▶ 💡 **SOLUTION**

---

# 5. `else` — When Everything Goes Right

The `else` block runs **only if no exception was raised** in the `try` block.

```
try:
    risky_code()
except SomeError:
    handle_error()
```

```
else:
    # only runs if try succeeded with no exception
    do_success_work()
```

## Why use `else` instead of just putting the code at the end of `try`?

Because code inside `try` is protected — any exception in it will be caught. Code inside `else` is **outside the protection**, so if it fails, it won't be silently swallowed by your `except` blocks. This keeps error handling precise.

```
# LESS PRECISE — success logic inside try means its errors are also caught
try:
    data = parse_input(raw)
    process_data(data)    # if this fails, except catches it too — was
that intended?
except ValueError:
    ...

# MORE PRECISE — only parse_input is protected
try:
    data = parse_input(raw)
except ValueError:
    ...
else:
    process_data(data)    # runs only on success, not protected by above
except
```

In [ ]:

```
# Topic: else block — precise control over success vs. error paths

def classify_alert(raw_score):
    """Parse and classify an alert severity from a raw CVSS string."""
    try:
        score = float(raw_score)           # only this is risky
    except (ValueError, TypeError) as e:
        print(f"[ERROR] Could not parse score '{raw_score}': {e}")
        return None
    else:
        # parse succeeded — classification logic is NOT inside the try block
        # so a bug here won't be silently swallowed by the except above
        if score >= 9.0:
            return "CRITICAL"
        elif score >= 7.0:
            return "HIGH"
        elif score >= 4.0:
            return "MEDIUM"
        else:
            return "LOW"


test_scores = ["9.8", "5.0", "not_a_score", "7.2"]
for raw in test_scores:
```

```
    severity = classify_alert(raw)
    if severity:
        print(f"Score '{raw}' -> {severity}")
```

---

# 6. `finally` — Always Runs, No Matter What

The `finally` block runs **always** — whether an exception occurred or not, whether it was caught or not, **even if there is a `return` or `raise` inside `try`**.

```
try:
    risky_code()
except SomeError:
    handle_error()
finally:
    # ALWAYS runs — perfect for cleanup
    close_connection()
    release_lock()
    log_attempt()
```

## When to use `finally`

Use `finally` for **cleanup that must happen regardless of outcome**:

| Scenario | What to put in `finally` |
|---|---|
| Network socket opened | `sock.close()` |
| File opened | `file.close()` |
| Database connection | `db.disconnect()` |
| Audit log | `log_attempt(username, success)` |
| Lock acquired | `lock.release()` |

In security tools, `finally` is critical for **audit logging** — you must record every authentication attempt, successful or not.

In [ ]:
```
# Topic: finally for audit logging — every attempt must be recorded

import time

def authenticate_user(username, provided_password, stored_hash):
    """
    Simulates password verification.
    The audit log entry must be written regardless of what happens.
    """
    audit_record = {
        "username": username,
```

```
        "timestamp": time.strftime("%Y-%m-%d %H:%M:%S"),
        "outcome": "UNKNOWN"
    }

    try:
        if not isinstance(provided_password, str):
            raise TypeError("Password must be a string")

        # Simulate hash comparison (real code would use bcrypt/argon2)
        if provided_password == stored_hash:
            audit_record["outcome"] = "SUCCESS"
            return True
        else:
            audit_record["outcome"] = "FAILURE"
            return False

    except TypeError as e:
        audit_record["outcome"] = f"ERROR: {e}"
        return False

    finally:
        # This ALWAYS runs — even when we return True or False above
        # Critical: authentication events must always be logged for compliance (SOC
        print(f"[AUDIT] {audit_record['timestamp']} | user={audit_record['username'


# Test all three paths
authenticate_user("alice",   "correct_hash", "correct_hash")  # SUCCESS
authenticate_user("bob",     "wrong_password", "correct_hash") # FAILURE
authenticate_user("charlie", 12345, "correct_hash")             # ERROR
```

## Practice

**Exercise 5 — Write**

Write a function `connect_to_siem(host, port)` that simulates a SIEM connection.

- In the `try` block: create a socket and attempt `connect_ex()` to `(host, port)`
- If `socket.gaierror` is raised (bad hostname), print an error message and set `connected = False`
- In `else`: if no error was raised, set `connected = True` and print a success message
- In `finally`: always print `'[AUDIT] Connection attempt to HOST:PORT logged'` and always call `sock.close()` if the socket was created

Test with `connect_to_siem('127.0.0.1', 514)` and `connect_to_siem('bad-siem-host', 514)`.

In [ ]:
```
# Your code here
import socket
```

▶ 💡 **SOLUTION**

---

# 7. `raise` — Triggering Exceptions Yourself

`raise` lets you **deliberately trigger an exception** when your code detects a problem. This is how you enforce rules and communicate errors upward to the caller.

```
raise ExceptionType("descriptive message")
```

## When to use `raise`

| Situation | What to raise |
|---|---|
| Invalid input value | `ValueError` |
| Wrong argument type | `TypeError` |
| Missing permission / access denied | `PermissionError` |
| State that should never happen | Custom exception (section 8) |

## Re-raising an exception

Sometimes you want to log an exception and then pass it up the call stack:

```
except ValueError as e:
    log_error(e)     # do something with it
    raise            # re-raise the SAME exception, preserving the
traceback
```

In [ ]:
```
# Topic: raise — enforcing input validation in a security function

def set_firewall_rule(action, protocol, port):
    """
    Adds a firewall rule. Raises exceptions for invalid inputs
    rather than silently accepting bad data.
    """
    # Validate action
    valid_actions = ["ALLOW", "DENY"]
    if action not in valid_actions:
        raise ValueError(
            f"Invalid action '{action}'. Must be one of: {valid_actions}"
        )

    # Validate protocol
    valid_protocols = ["TCP", "UDP", "ICMP"]
    if protocol not in valid_protocols:
        raise ValueError(
            f"Invalid protocol '{protocol}'. Must be one of: {valid_protocols}"
        )
```

```
    # Validate port range
    if not isinstance(port, int):
        raise TypeError(f"Port must be an int, got {type(port).__name__}")
    if not 1 <= port <= 65535:
        raise ValueError(f"Port {port} is out of range (1-65535)")

    # If we reach here, all inputs are valid
    print(f"[OK] Rule added: {action} {protocol}/{port}")
    return True


# Test valid rule
set_firewall_rule("DENY", "TCP", 23)     # Telnet — should be denied

# Test invalid inputs — each should raise and be caught
test_cases = [
    ("PERMIT", "TCP", 80),    # invalid action
    ("ALLOW", "TCP", 99999),  # port out of range
    ("DENY", "TCP", "443"),   # port as string, not int
]

for action, protocol, port in test_cases:
    try:
        set_firewall_rule(action, protocol, port)
    except (ValueError, TypeError) as e:
        print(f"[REJECTED] {e}")
```

## Practice

**Exercise 6 — Write**

Write a function `register_user(username, password)` that raises exceptions for:

- `ValueError` if `username` is empty or contains spaces
- `ValueError` if `password` is shorter than 12 characters
- `ValueError` if `password` does not contain at least one digit

If all checks pass, return `True`.

Test with valid and invalid inputs, catching the exceptions.

In [ ]:
```
# Your code here
```

▶ 💡 **SOLUTION**

---

# 8. Custom Exceptions — Building Your Own Error Types

Built-in exceptions like `ValueError` and `TypeError` are generic. For security tools, **custom exceptions** let you:

- Give errors meaningful, domain-specific names
- Attach extra context (IP, port, CVE ID, severity)
- Catch specific security errors without catching unrelated `ValueError`s

Creating a custom exception is simple — just subclass `Exception`:

```
class MyCustomError(Exception):
    pass          # inherits everything from Exception
```

You can also add extra data:

```
class MyCustomError(Exception):
    def __init__(self, message, extra_info):
        super().__init__(message)     # the standard message
        self.extra_info = extra_info  # your custom attribute
```

In [ ]:

```
# Topic: Custom exceptions for a security tool

class SecurityError(Exception):
    """Base class for all security-related exceptions in this tool.
    Catching SecurityError catches all subclasses below.
    """
    pass


class UnauthorisedAccessError(SecurityError):
    """Raised when a user attempts an action they are not permitted to perform."""

    def __init__(self, username, attempted_action, required_role):
        message = (
            f"User '{username}' attempted '{attempted_action}' "
            f"but requires role: {required_role}"
        )
        super().__init__(message)
        self.username = username
        self.attempted_action = attempted_action
        self.required_role = required_role


class MaliciousInputError(SecurityError):
    """Raised when input contains patterns that look like an attack."""

    def __init__(self, input_value, pattern_matched):
        message = f"Malicious pattern '{pattern_matched}' detected in input"
        super().__init__(message)
        self.input_value = input_value
        self.pattern_matched = pattern_matched


def check_input(user_input):
    """Simple input sanitation — detects common SQL injection markers."""
    sql_patterns = ["' OR '", "DROP TABLE", "--", "1=1"]
```

```
    for pattern in sql_patterns:
        if pattern.upper() in user_input.upper():
            raise MaliciousInputError(user_input, pattern)
    return user_input


def delete_user(requesting_user_role, target_username):
    if requesting_user_role != "admin":
        raise UnauthorisedAccessError(
            username="jdoe",
            attempted_action="delete_user",
            required_role="admin"
        )
    print(f"[OK] User {target_username} deleted")


# Test both custom exceptions
for test in [
    lambda: check_input("SELECT * FROM users WHERE '1'='1'"),
    lambda: delete_user("analyst", "alice"),
    lambda: check_input("normal_username"),
]:
    try:
        result = test()
        print(f"[OK] Input accepted: {result}")
    except MaliciousInputError as e:
        print(f"[SECURITY ALERT] {e} | pattern: {e.pattern_matched}")
    except UnauthorisedAccessError as e:
        print(f"[SECURITY ALERT] {e} | required: {e.required_role}")
    except SecurityError as e:
        # Catch-all for any other security exception
        print(f"[SECURITY ERROR] {e}")
```

## Practice

### Exercise 7 — Write

Create two custom exceptions:

1. `InvalidCvssError(Exception)` — raised when a CVSS score is outside 0.0-10.0. Store the invalid `score` as an attribute.
2. `UnknownCveError(Exception)` — raised when a CVE ID doesn't match the format `CVE-YYYY-NNNNN`. Store the invalid `cve_id` as an attribute.

Write a function `validate_vulnerability(cve_id, cvss_score)` that raises the appropriate exception for invalid inputs, and returns `True` if both are valid.

Test with: `('CVE-2023-44487', 9.8)` (valid), `('MS-2023-001', 7.5)` (bad CVE), `('CVE-2021-44228', 15.0)` (bad score).

In [ ]:
```
# Your code here
```

▶ 💡 **SOLUTION**

---

# 9. The Exception Hierarchy

Python's exceptions form a tree. Understanding this helps you decide what to catch.

```
BaseException
 +-- SystemExit              <- sys.exit() — do NOT catch this
normally
 +-- KeyboardInterrupt       <- Ctrl+C — do NOT catch this normally
 +-- Exception               <- catch THIS as your broadest safety
net
      +-- ValueError
      +-- TypeError
      +-- OSError
      |    +-- FileNotFoundError
      |    +-- PermissionError
      |    +-- ConnectionError
      |         +-- ConnectionRefusedError
      |         +-- ConnectionTimeoutError
      +-- KeyError
      +-- IndexError
      +-- ZeroDivisionError
      +-- AttributeError
```

## Key rules

| Rule | Why |
|---|---|
| Catch `Exception`, not `BaseException` | `BaseException` catches `SystemExit` and `KeyboardInterrupt` — you almost never want that |
| Never use bare `except:` | Same problem as `BaseException`, plus it hides bugs |
| Catching a parent catches all children | `except OSError` catches `FileNotFoundError`, `PermissionError`, `ConnectionError` etc. |
| Most specific first | Put `FileNotFoundError` before `OSError` — Python stops at the first match |

In [ ]:
```python
# Topic: Exception hierarchy in practice — handling connection errors by specificit

import socket

def probe_target(target_ip, port):
    """Demonstrate exception hierarchy — specific errors before general ones."""
    try:
        socket.setdefaulttimeout(0.5)
        sock = socket.socket()
```

```
        sock.connect((target_ip, port))
        sock.close()
        return "OPEN"

    except socket.gaierror:
        # Subclass of OSError — name could not be resolved
        return "UNRESOLVABLE"

    except ConnectionRefusedError:
        # Subclass of ConnectionError -> OSError — host exists but port is closed
        return "REFUSED"

    except socket.timeout:
        # Timeout — likely filtered by a firewall
        return "FILTERED"

    except OSError as e:
        # Catches any OTHER OS-level error not handled above
        return f"OS_ERROR: {e}"


print(probe_target("127.0.0.1", 9)        )  # port 9 (discard) — likely refused
print(probe_target("invalid-host-xyz", 80))  # unresolvable
print(probe_target("127.0.0.1", 80)       )  # depends on local setup
```

---

# 10. Summary and Next Steps

## The full structure

```
try:
    risky_code()              # code that might raise an exception
except SpecificError as e:    # most specific first — catches + names it
    handle_specific(e)        # what to do if this specific error occurs
except (OtherA, OtherB):     # group related errors
    handle_others()
except Exception as e:        # safety net — catches anything else
    handle_unexpected(e)
else:
    success_work()            # runs ONLY if no exception was raised
finally:
    cleanup()                 # ALWAYS runs — close sockets, write audit
logs
```

## Key rules to remember

| Rule | Details |
|---|---|
| Always name the exception | `except ValueError:` not `except:` |
| Use `as e` for logging | Always log *what* went wrong, not just *that* it went wrong |

| Rule | Details |
|---|---|
| `finally` for cleanup | Sockets, files, locks, audit logs |
| `else` for success logic | Keeps error scope precise |
| Most specific first | `FileNotFoundError` before `OSError` |
| Custom exceptions | Use them in security tools for meaningful, catchable errors |
| Re-raise when needed | `raise` alone inside except re-raises the original |

## Next Steps

- **01_Drills_Exception_Handling.ipynb** — 15 exercises to practice all patterns